POO COM JAVA

Pedro Henrique Ribeiro Dias

Inatel

INTRODUÇÃO

Diferente de linguagens criadas para flexibilidade ou velocidade de escrita, o Java foi construído com um objetivo claro – rodar em qualquer lugar. Lançado em 1995 com o slogan "Write Once, Run Anywhere", ele foi projetado para ser robusto, seguro e previsível, características que o tornaram a espinha dorsal de sistemas bancários, governamentais e empresariais ao redor do mundo.

O que torna o Java único?

- Portabilidade: O código Java não é compilado para uma máquina específica, mas para a JVM – a Java Virtual Machine. Isso significa que o mesmo programa roda no Windows, Linux ou Mac sem nenhuma alteração.
- Tipagem Forte: Java exige que você declare o tipo de cada variável antes de usá-la. Isso parece verboso no início, mas elimina uma classe inteira de erros em tempo de execução – o compilador pega o problema antes do programa rodar.
- Orientação a Objetos de Verdade: Diferente de linguagens onde POO é opcional, em Java tudo existe dentro de uma classe. Não há funções soltas, não há scripts avulsos – a estrutura é parte da linguagem.

PILARES DA POO

1. Abstração
2. Herança
3. Encapsulamento
4. Polimorfismo

RELACIONAMENTOS

- Composição: A classe cria e controla o objeto interno.
- Agregação: A classe recebe o objeto pronto.

Ou seja:

- 🔸 Composição → dependência total
- 🔹 Agregação → ligação fraca (referência)

ARRAY LIST

1. Sequência ordenada de elementos. Permite elementos duplicados. Acesso rápido a elementos por índice (.get(i)). Listas de itens onde a ordem de inserção é importante (ex: histórico de pedidos, lista de alunos por chamada).

   Exemplo: ArrayList<String> nomes = new ArrayList<>();

HASH SET

1. O HashSet é a implementação mais performática da interface Set. Sua principal característica é não permitir elementos duplicados, garantindo a unicidade de cada item. Ele não mantém a ordem de inserção, pois a organização interna é baseada no hash code dos objetos. É a escolha ideal para coleções onde apenas a presença única do elemento importa. Exemplo de declaração: HashSet<Integer> ids = new HashSet<>();

HASH MAP

1. O HashMap armazena dados no formato chave-valor, sendo a implementação mais comum da interface Map. A chave deve ser única, servindo como um índice ultrarrápido para recuperar o valor associado (.get(chave)). O acesso e a manipulação são extremamente rápidos, mas a ordem de inserção não é preservada. É perfeito para mapeamentos e dicionários (ex: CPF: Pessoa).
Exemplo de declaração: HashMap<String, Livro> catalogo = new HashMap<>();

INTERFACES E CLASSES ABSTRATAS

- Java tem suporte nativo tanto para interfaces quanto para classes abstratas – são duas formas diferentes de definir contratos.
- A interface é criada com interface e define apenas o que uma classe deve ter, sem nenhuma implementação. A classe que assinar esse contrato usa implements e é obrigada a implementar todos os métodos.
- A classe abstrata é criada com abstract e fica no meio do caminho – ela pode ter métodos implementados e métodos abstratos ao mesmo tempo. A classe filha usa extends e é obrigada a implementar apenas os métodos marcados como abstract.
- A diferença prática: use interface quando classes sem nenhuma relação precisam compartilhar um contrato. Use classe abstrata quando as classes filhas compartilham uma base comum mas precisam sobrescrever partes específicas.

EXERCÍCIO 1

Cenário: Em O Castelo Animado, filme do Studio Ghibli, um castelo mágico anda sozinho pelos campos, movido por um **demônio do fogo** que vive na lareira. Lá moram **magos** e o próprio demônio, e todo morador sabe se apresentar, cada um do seu jeito. Não existe um "morador genérico": todo morador é de um tipo específico.

Tarefa:

1. Crie a classe abstrata Morador:
   - atributo private String nome, recebido no construtor;
   - getter public String getNome();
   - método abstrato public abstract void apresentar();.
2. Crie a classe Mago, que herda de Morador:
   - atributo private String feitico;
   - construtor Mago(String nome, String feitico), que repassa o nome com super(nome);
   - sobrescreva apresentar() com @Override, imprimindo o nome (use getNome()) e o feitiço

EXERCÍCIO 1

3. Crie a classe DemonioDoFogo, que herda de Morador:
    - construtor DemonioDoFogo(String nome), que repassa o nome com super(nome);
    - sobrescreva apresentar() com @Override, imprimindo o nome e que ele faz o castelo andar.
4. Na main, crie um Mago e um DemonioDoFogo, à sua escolha, guardando os dois em variáveis do tipo Morador. Chame apresentar() nos dois.
5. Tente criar um objeto Morador diretamente no main e observe o erro: o programa nem chega a compilar. Depois, transforme essa linha em comentário (//) para o programa voltar a rodar.

EXERCÍCIO 2

Cenário: Em Persona 5, o Leblanc é um pequeno café em Tóquio, famoso pelo café coado na hora e pelo curry da casa. O cardápio tem itens comuns, vendidos pelo preço cadastrado, e **combos**, que saem com desconto.

Tarefa:

1. Crie a classe ItemDoCardapio:
   - atributos private String nome e private double preco, recebidos no construtor;
   - getters public String getNome() e public double getPreco();
   - método public double precoFinal(), que retorna o preço.
2. Crie a classe ComboLeblanc, que herda de ItemDoCardapio:
   - construtor ComboLeblanc(String nome, double preco), que repassa os dois valores com super(nome, preco);
   - sobrescreva precoFinal() com @Override, retornando o preço com 10% de desconto (getPreco() * 0.9).

EXERCÍCIO 2

3. **Crie a classe CafeLeblanc:**
   - **atributo private ArrayList<ItemDoCardapio> cardapio, criado vazio no construtor, que não recebe parâmetros;**
   - **método public void adicionar(ItemDoCardapio item), que adiciona um item já criado ao cardápio;**
   - **método public void mostrarCardapio(), que percorre o cardápio com um for-each e imprime o nome e o precoFinal() de cada item.**
4. **No main, crie dois ItemDoCardapio e um ComboLeblanc, com nomes e preços à sua escolha. Crie o CafeLeblanc, adicione os três itens e chame mostrarCardapio().**

EXERCÍCIO 3

Cenário: Em Dandadan, dois estudantes do ensino médio descobrem que fantasmas e alienígenas existem de verdade e passam a anotar, em um mapa, os locais da cidade onde cada ser sobrenatural foi visto. Alienígenas e espíritos não têm nada em comum, a não ser uma coisa: todos se **manifestam** quando alguém investiga o local.

Tarefa:

1. Interface: crie a interface Sobrenatural, com o método void manifestar();.
2. Crie a classe Alienigena, que implementa Sobrenatural:
   - atributo private String nome, recebido no construtor;
   - implemente public void manifestar() com @Override, imprimindo o nome e um comportamento de alienígena à sua escolha.
3. Crie a classe EspiritoYoukai, que também implementa Sobrenatural, do mesmo jeito: atributo private String nome, recebido no construtor, e public void manifestar() com um comportamento de espírito à sua escolha.

EXERCÍCIO 3

4. Crie a classe MapaDeAvistamentos:
   - atributo private HashMap<String, Sobrenatural> avistamentos, criado vazio no construtor, que não recebe parâmetros; a chave do mapa é o nome do local;
   - método public void registrar(String local, Sobrenatural ser), que guarda o ser no mapa, usando o local como chave;
   - método public void investigar(String local), que busca o local no mapa: se não encontrar nada (null), imprime que o local está tranquilo; se encontrar, chama manifestar() do ser.
5. Na main, crie um Alienigena e um EspiritoYoukai, à sua escolha, e registre cada um em um local à sua escolha. Depois, investigue os dois locais e um terceiro local, que não foi registrado.

EXERCÍCIO 4

Cenário: Em Dungeon Meshi (Delicious in Dungeon), um grupo de aventureiros explora uma masmorra e, sem comida, aprende com um cozinheiro anão a preparar os monstros que derrota. Todo monstro vira ingrediente, mas cada tipo tem a sua própria receita. O cozinheiro carrega sempre a sua panela, que é dele e nasce com ele (composição), e junta os monstros que o grupo derrota pelo caminho (agregação).

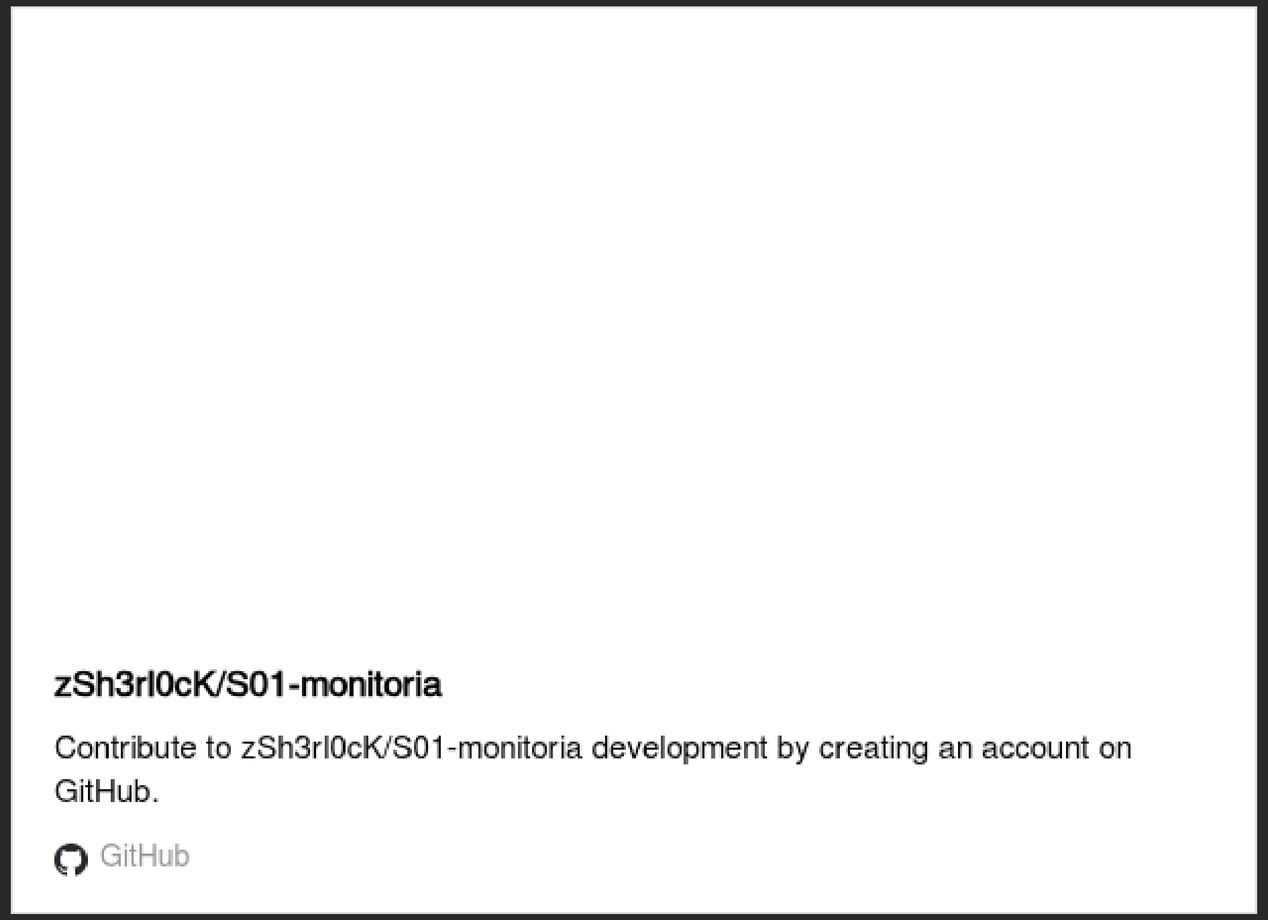
Tarefa:

1. Crie a classe abstrata Monstro:
   - atributo private String nome, recebido no construtor, e o getter public String getNome();
   - método abstrato public abstract String receita();, que nas filhas vai retornar o modo de preparo.
2. Crie as classes Slime e Mandragora, que herdam de Monstro. Cada uma tem um construtor (Slime(String nome) e Mandragora(String nome)) que repassa o nome com super(nome), e sobrescreve receita() com @Override, retornando um modo de preparo à sua escolha.

EXERCÍCIO 4

3. Parte composta: crie a classe Panela, com o atributo private String material, recebido no construtor, e o getter public String getMaterial().
4. Crie a classe Cozinheiro:
    - atributos private String nome, private Panela panela e private ArrayList<Monstro> ingredientes;
    - construtor Cozinheiro(String nome, String materialDaPanela), que cria a Panela dentro dele, com o material recebido, e cria a lista de ingredientes vazia;
    - método public void adicionarIngrediente(Monstro monstro), que adiciona um monstro já criado à lista;
    - método public void servirBanquete(), que imprime o nome do cozinheiro e o material da panela e, com um for-each, o nome e a receita() de cada ingrediente.
5. No main, crie um Slime e uma Mandragora, à sua escolha, antes do cozinheiro. Depois, crie o cozinheiro, com o nome e o material da panela à sua escolha, adicione os dois monstros e chame servirBanquete().
6. Explique, em um comentário no código, o que é composição e o que é agregação no seu programa.

FIM DA AULA!


zSh3rl0cK/S01-monitoria
Contribute to zSh3rl0cK/S01-monitoria development by creating an account on GitHub.
GitHub